

DBMS

COMPLETE NOTES

Easy Language • Real-Life Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to DBMS

- 2. Types of Databases (Relational vs Non-Relational)
- 3. Data Models

★ 4. Keys in DBMS (Primary, Foreign, Candidate)

★ 5. Normalization

★ 6. ER Diagrams (Entity-Relationship Diagrams)

- 7. Relational Algebra

★ 8. Transactions

★ 9. ACID Properties

- 10. Concurrency Control

★ 11. Indexing

★ 12. Joins

- 13. Views
- 14. Deadlocks
- 15. Data Integrity Constraints
- 16. Backup and Recovery

★ 1. Introduction to DBMS [IMPORTANT TOPIC]

Explanation:

A DBMS (Database Management System) is software used to store, organize, and manage data efficiently, allowing multiple users and programs to access it safely at the same time. Instead of storing data in simple files, a DBMS organizes it in a structured, searchable way.

Key Points:

- Manages large amounts of data efficiently and safely.
- Allows multiple users to access data at the same time.
- Provides security, backup, and data recovery features.
- Examples: MySQL, Oracle, PostgreSQL, MongoDB.

Example:

A college uses a DBMS to store thousands of student records, allowing staff, teachers, and students to access relevant information at the same time, safely and quickly.

Real-Life Example:

Every time you check your bank balance on a banking app, a DBMS in the background is retrieving your exact account data instantly.

2. Types of Databases (Relational vs Non-Relational)

Explanation:

A Relational Database (SQL) stores data in structured TABLES with rows and columns, and relationships between tables. A Non-Relational Database (NoSQL) stores data more flexibly, often as documents or key-value pairs, without a fixed table structure.

Key Points:

- Relational (SQL) -> MySQL, PostgreSQL, Oracle — best for structured, related data.
- Non-Relational (NoSQL) -> MongoDB, Redis — best for flexible, fast-changing data.

Example:

A banking system (with strict, structured relationships between accounts and transactions) typically uses a Relational database. A social media app storing varied user posts often uses a NoSQL database.

Real-Life Example:

Facebook and Instagram use NoSQL databases to handle huge amounts of flexible, fast-changing content like posts and comments.

3. Data Models

Explanation:

A Data Model defines HOW data is logically structured and organized within a database — it is the blueprint for how information relates to other information.

Key Points:

- Relational Model -> data organized into tables (most common today).
- Hierarchical Model -> data organized in a tree-like parent-child structure.
- Network Model -> data organized as a graph with multiple relationships.

Example:

A university database using the Relational Model would have separate tables for Students, Courses, and Enrollments, all linked together.

Real-Life Example:

The Relational Model, introduced in the 1970s, is still the most widely used data model in real-world business software today.

★ 4. Keys in DBMS (Primary, Foreign, Candidate) [IMPORTANT TOPIC]

Explanation:

Keys are special columns used to uniquely identify rows and to connect different tables together.

Key Points:

- Primary Key -> uniquely identifies each row in a table; cannot be duplicated or empty.
- Candidate Key -> any column (or combination) that COULD have been chosen as the primary key.
- Foreign Key -> a column that links to the Primary Key of ANOTHER table, creating a relationship.

Example:

In a Students table, 'student_id' is the Primary Key. In an Enrollments table, 'student_id' appears again as a Foreign Key, linking each enrollment back to a specific student.

Real-Life Example:

Your Aadhaar number acts like a real-world Primary Key — a unique identifier that no two people share.

★ 5. Normalization [IMPORTANT TOPIC]

Explanation:

Normalization is the process of organizing a database's tables to REDUCE duplicate data and avoid data problems (called 'anomalies'), by breaking large tables into smaller, well-structured related tables.

Key Points:

- 1NF (First Normal Form) -> each column holds only ONE value (no lists inside a cell).
- 2NF (Second Normal Form) -> removes data that only depends on PART of a key.
- 3NF (Third Normal Form) -> removes data that depends on a NON-KEY column instead of the actual key.

Example:

Instead of storing a student's course name repeatedly in every single row, normalization splits this into a separate Courses table, connected using a course_id — avoiding repeated, duplicate data.

Real-Life Example:

Poorly normalized databases are a common real-world cause of data getting out of sync — like a customer's address showing differently on different pages of the same app.

★ 6. ER Diagrams (Entity-Relationship Diagrams) [IMPORTANT TOPIC]

Explanation:

An ER Diagram is a visual drawing used to PLAN a database before building it — showing 'entities' (things like Student, Course), their attributes (details like name, age), and the relationships between them.

Key Points:

- Entity -> a real-world object or concept (example: Student, Product).
- Attribute -> a property/detail of an entity (example: name, price).
- Relationship -> how two entities are connected (example: a Student ENROLLS IN a Course).

Example:

Before building a hospital database, a designer draws an ER diagram showing entities like Patient, Doctor, and Appointment, and how they connect to each other.

Real-Life Example:

ER diagrams are a standard first step taught and used in real software companies before any database is actually created.

7. Relational Algebra

Explanation:

Relational Algebra is a formal, mathematical way of describing operations on relational database tables — like selecting rows, choosing columns, or combining tables. SQL queries are actually based on these underlying operations.

Key Points:

- Selection (σ) -> picks rows matching a condition.
- Projection (π) -> picks specific columns.
- Union, Intersection -> combine results from two queries.
- Join -> combines data from two related tables.

Example:

The relational algebra operation $\sigma(\text{age} > 21)(\text{Students})$ means: select all rows from the Students table where age is greater than 21 — this is exactly what a SQL WHERE clause does.

Real-Life Example:

Relational algebra is the theoretical foundation that SQL query engines use internally to process and optimize queries.

★ 8. Transactions [IMPORTANT TOPIC]

Explanation:

A Transaction is a group of one or more database operations that are treated as ONE single unit — they either ALL complete successfully together, or NONE of them happen at all, keeping the data safe and consistent.

Example:

Transferring money between two bank accounts involves TWO steps: subtracting from one account and adding to another. A transaction ensures BOTH happen together, or neither happens, even if the system crashes in between.

Real-Life Example:

Every online payment or bank transfer relies on database transactions to make sure money is never lost or duplicated due to a system failure.

★ 9. ACID Properties [IMPORTANT TOPIC]

Explanation:

ACID is a set of four properties that guarantee a database transaction is safe and reliable, even if something goes wrong in the middle.

Key Points:

- Atomicity -> a transaction either FULLY completes, or doesn't happen at all.
- Consistency -> the database always moves from one VALID state to another.
- Isolation -> transactions running at the same time don't interfere with each other.
- Durability -> once completed, a transaction's changes are PERMANENTLY saved, even after a crash.

Example:

During a money transfer, Atomicity ensures both the debit and credit happen together; Durability ensures that once confirmed, the transfer stays saved even if the server crashes seconds later.

Real-Life Example:

Banking systems worldwide are legally required to follow ACID-compliant database practices to protect customer money.

10. Concurrency Control

Explanation:

Concurrency Control manages what happens when MULTIPLE users try to access or change the SAME data at the same time, preventing conflicts and keeping the data correct and consistent.

Example:

Two people try to book the LAST available movie seat at the exact same second. Concurrency control ensures only ONE booking actually succeeds, preventing the seat from being sold twice.

Real-Life Example:

E-commerce sites like Amazon rely heavily on concurrency control during big sales, when thousands of people might try to buy the same limited-stock item simultaneously.

★ 11. Indexing [IMPORTANT TOPIC]

Explanation:

An index is a special data structure that dramatically speeds up SEARCHING for data in a table, similar to an index at the back of a book that helps you find a topic without reading every page.

Example:

Searching for a student by name in a table of 10 million rows would normally check every row one by one. With an index on the 'name' column, the database can find the exact row almost instantly instead.

Real-Life Example:

Search engines like Google use similar indexing concepts internally to return search results in a fraction of a second from billions of web pages.

 *Note: Indexes make reading data faster, but they slightly slow down writing (INSERT/UPDATE), since the index itself must also be updated.*

★ 12. Joins *[IMPORTANT TOPIC]*

Explanation:

A Join combines related data from TWO OR MORE tables into a single result, based on a common column between them — essential since normalized databases split data into multiple related tables.

Example:

A Students table and a Courses table can be JOINED using a shared 'course_id' column, to produce a single combined report showing each student's name alongside their enrolled course name.

Real-Life Example:

Almost every real-world report — like a sales report combining Customer, Product, and Order tables — relies on joins to bring related data together.

13. Views

Explanation:

A View is a saved, virtual table based on a specific query. It doesn't store data on its own, but provides a simplified, reusable way to look at data from one or more real tables.

Example:

A company creates a view called 'ActiveEmployees' that always shows only employees who are currently working, hiding the complexity of the underlying query from other staff who use it.

Real-Life Example:

Views are often used to give certain staff limited, simplified access to only the data relevant to their job, without exposing the entire database structure.

14. Deadlocks

Explanation:

A Deadlock happens when two or more transactions are each waiting for the OTHER to release a resource (like a locked row), and neither can proceed — resulting in both being stuck forever unless the system intervenes.

Example:

Transaction A locks Row 1 and waits for Row 2. Transaction B locks Row 2 and waits for Row 1. Both wait forever for each other — a classic deadlock.

Real-Life Example:

Modern database systems automatically detect deadlocks and forcibly cancel (rollback) one of the transactions to let the other continue safely.

15. Data Integrity Constraints

Explanation:

Constraints are rules applied to table columns to make sure only VALID, correct data can be stored, preventing mistakes before they happen.

Key Points:

- NOT NULL -> a column cannot be left empty.
- UNIQUE -> no two rows can have the same value in this column.
- CHECK -> a column's value must satisfy a specific condition.
- DEFAULT -> automatically sets a value if none is provided.

Example:

A CHECK constraint like age ≥ 18 on a 'voters' table automatically prevents anyone from accidentally being added with an invalid age.

Real-Life Example:

Data integrity constraints are the reason a website form rejects an invalid email format or a duplicate username before it even reaches the database.

16. Backup and Recovery

Explanation:

Backup means regularly saving a COPY of the database, so data isn't permanently lost if something goes wrong. Recovery is the process of RESTORING the database back to a working state after a failure, crash, or accidental deletion.

Example:

A company takes a full database backup every night. If the server crashes on Wednesday, they can recover the database using Tuesday night's backup, losing only that one day's changes.

Real-Life Example:

Major cloud providers like AWS and Google Cloud offer automated daily database backups specifically to protect businesses from catastrophic data loss.